

### **Prometheus thu thập metric từ microservices như thế nào?**

[VN] Prometheus thu thập metric từ microservices bằng pull model. Mỗi service sẽ expose một endpoint (thường là /metrics) chứa các metric như request count, latency hoặc error rate. Prometheus được cấu hình để định kỳ gửi request đến endpoint này và scrape dữ liệu metric. Sau đó Prometheus lưu các metric này dưới dạng time-series trong database của nó để phục vụ việc phân tích, alerting và hiển thị trên dashboard như Grafana.

[EN] Prometheus collects metrics from microservices using a pull model. Each service exposes an endpoint (usually /metrics) that contains metrics such as request count, latency, or error rate. Prometheus is configured to periodically send requests to this endpoint and scrape the metric data. The collected metrics are then stored as time-series data in Prometheus and can be used for analysis, alerting, and visualization in dashboards like Grafana.

### **Trong Kubernetes, Prometheus discover service để scrape metric bằng cách nào?**

[VN] Trong môi trường Kubernetes, Prometheus có thể tự động discover các service để scrape metric thông qua cơ chế service discovery. Prometheus thường sử dụng Kubernetes API để tìm các pod hoặc service đang chạy trong cluster. Các service có thể được đánh dấu bằng annotations hoặc được quản lý thông qua ServiceMonitor/PodMonitor (khi dùng Prometheus Operator). Khi phát hiện service có endpoint /metrics, Prometheus sẽ định kỳ gửi request đến endpoint đó để thu thập metric.

[EN] In a Kubernetes environment, Prometheus can automatically discover services to scrape metrics using service discovery. Prometheus typically uses the Kubernetes API to find running pods or services in the cluster. Services can be marked using annotations or managed through ServiceMonitor/PodMonitor (when using Prometheus Operator). When a service exposes a /metrics endpoint, Prometheus periodically sends requests to that endpoint to collect metrics.

### **Grafana lấy dữ liệu từ Prometheus như thế nào để hiển thị dashboard?**

[VN] Grafana không tự thu thập dữ liệu mà lấy dữ liệu từ các data source như Prometheus. Trong hệ thống monitoring, Grafana được cấu hình kết nối đến Prometheus như một data source. Khi tạo dashboard, Grafana sẽ gửi các PromQL query đến Prometheus để lấy metric (ví dụ request rate, latency hoặc CPU usage). Sau đó Grafana nhận dữ liệu time-series từ Prometheus và hiển thị dưới dạng biểu đồ, bảng hoặc các visualization khác trên dashboard.

[EN] Grafana does not collect data by itself; it retrieves data from data sources such as Prometheus. In a monitoring system, Grafana is configured to connect to Prometheus as a

data source. When building a dashboard, Grafana sends PromQL queries to Prometheus to retrieve metrics (for example request rate, latency, or CPU usage). Prometheus returns the time-series data, and Grafana visualizes it as charts, tables, or other dashboard panels.

### **Metric, log và trace khác nhau như thế nào trong monitoring system?**

[VN] Trong một hệ thống monitoring, metric, log và trace là ba loại dữ liệu quan trọng để quan sát hệ thống. Metric là dữ liệu dạng số theo thời gian, dùng để đo hiệu năng hệ thống như CPU usage, request rate hoặc latency và thường được thu thập bởi các công cụ như Prometheus. Log là các bản ghi chi tiết về sự kiện xảy ra trong hệ thống, ví dụ như lỗi, request hoặc thông tin debug. Trace dùng để theo dõi toàn bộ đường đi của một request qua nhiều service trong hệ thống microservices, giúp tìm ra service nào gây chậm hoặc lỗi. Các công cụ như Grafana thường hiển thị metric và có thể tích hợp với hệ thống log và trace để giúp debug hệ thống hiệu quả hơn.

[EN] In a monitoring system, metrics, logs, and traces are three important types of data used to observe system behavior. Metrics are numerical data over time used to measure system performance, such as CPU usage, request rate, or latency, and are often collected by tools like Prometheus. Logs are detailed records of events happening in the system, such as errors, requests, or debug information. Traces track the full path of a request as it travels through multiple services in a microservices architecture, helping identify which service causes delays or errors. Tools like Grafana typically visualize metrics and can integrate with logging and tracing systems to help diagnose issues more effectively.

### **Trong một hệ thống banking transaction, bạn sẽ monitor những metric nào quan trọng?**

[VN] Trong một hệ thống banking transaction, cần monitor các metric quan trọng để đảm bảo giao dịch được xử lý nhanh và ổn định. Ví dụ, request rate để biết số lượng request hoặc transaction mỗi giây, latency để đo thời gian xử lý giao dịch, và error rate để phát hiện các lỗi xảy ra trong hệ thống. Ngoài ra cũng cần theo dõi CPU và memory usage của service, database response time, và các metric của message queue như queue length hoặc consumer lag. Những metric này thường được thu thập bởi Prometheus và hiển thị trên dashboard của Grafana để giúp team nhanh chóng phát hiện và xử lý sự cố.

[EN] In a banking transaction system, it is important to monitor key metrics to ensure transactions are processed quickly and reliably. For example, request rate shows the number of requests or transactions per second, latency measures the processing time of transactions, and error rate helps detect failures in the system. It is also important to monitor CPU and memory usage of services, database response time, and message queue metrics such as queue length or consumer lag. These metrics are typically collected by

Prometheus and visualized on dashboards in Grafana to help teams quickly detect and troubleshoot issues.

### **Làm sao expose metric từ Golang service cho Prometheus?**

[VN] Để expose metric từ một service viết bằng Go cho Prometheus, thường sử dụng thư viện Prometheus client cho Go. Trong service, ta định nghĩa các metric như counter, gauge hoặc histogram bằng thư viện này, sau đó đăng ký một HTTP endpoint (thường là /metrics). Endpoint này sẽ trả về tất cả metric của service theo định dạng mà Prometheus hiểu được. Prometheus sau đó sẽ định kỳ gửi request đến endpoint /metrics để scrape và lưu dữ liệu metric.

[EN] To expose metrics from a service written in Go to Prometheus, developers typically use the Prometheus client library for Go. In the service, metrics such as counters, gauges, or histograms are defined using this library, and an HTTP endpoint (usually /metrics) is created. This endpoint returns all service metrics in a format that Prometheus can understand. Prometheus then periodically sends requests to the /metrics endpoint to scrape and store the metric data.

### **Histogram và Counter trong Prometheus dùng khi nào?**

[VN] Trong Prometheus, Counter và Histogram là hai loại metric dùng cho các mục đích khác nhau. Counter được dùng để đếm các giá trị chỉ tăng theo thời gian, ví dụ như tổng số request, tổng số transaction hoặc số lỗi xảy ra trong hệ thống. Counter chỉ có thể tăng hoặc reset khi service restart. Histogram được dùng để đo phân bố của dữ liệu, thường dùng để theo dõi thời gian xử lý request (latency). Histogram chia dữ liệu thành nhiều khoảng giá trị (buckets) để biết bao nhiêu request rơi vào từng khoảng thời gian, từ đó có thể phân tích hiệu năng của hệ thống.

[EN] In Prometheus, Counter and Histogram are two types of metrics used for different purposes. A Counter is used to count values that only increase over time, such as the total number of requests, total transactions, or number of errors in a system. A counter can only increase or reset when the service restarts. A Histogram is used to measure the distribution of values, and it is commonly used to track request processing time (latency). A histogram groups values into multiple ranges called buckets, which helps analyze system performance.

### **Alerting trong Prometheus hoạt động như thế nào?**

[VN] Trong Prometheus, alerting hoạt động bằng cách định nghĩa các alert rules dựa trên metric. Prometheus sẽ liên tục kiểm tra các rule này bằng cách chạy các truy vấn PromQL trên dữ liệu metric. Khi điều kiện trong rule được thỏa mãn (ví dụ error rate quá cao hoặc

CPU usage vượt ngưỡng), Prometheus sẽ tạo một alert và gửi nó đến Alertmanager. Alertmanager sau đó sẽ xử lý alert này như nhóm alert, lọc hoặc gửi thông báo đến các kênh như email, Slack hoặc webhook để thông báo cho team vận hành.

[EN] In Prometheus, alerting works by defining alert rules based on metrics. Prometheus continuously evaluates these rules by running PromQL queries on the collected metric data. When a rule condition is met (for example, a high error rate or CPU usage above a threshold), Prometheus triggers an alert and sends it to Alertmanager. Alertmanager then processes the alert by grouping, filtering, and sending notifications to channels such as email, Slack, or webhooks to notify the operations team.

### **Grafana dashboard thường dùng để visualize những metric gì trong production?**

[VN] Trong môi trường production, dashboard của Grafana thường được dùng để visualize các metric quan trọng của hệ thống nhằm theo dõi hiệu năng và phát hiện sự cố nhanh chóng. Ví dụ như request rate (số request mỗi giây), latency (thời gian xử lý request), error rate (tỷ lệ lỗi), và các metric về tài nguyên như CPU và memory usage. Ngoài ra còn có thể theo dõi database query time, queue length hoặc consumer lag của message queue, và trạng thái của các service trong hệ thống. Những metric này thường được thu thập từ Prometheus và hiển thị trên Grafana để giúp team vận hành dễ dàng quan sát và phân tích hệ thống.

[EN] In production environments, dashboards in Grafana are commonly used to visualize important system metrics in order to monitor performance and detect issues quickly. Examples include request rate (number of requests per second), latency (request processing time), error rate, and resource metrics such as CPU and memory usage. Dashboards may also show database query time, message queue metrics like queue length or consumer lag, and the health status of services in the system. These metrics are usually collected from Prometheus and displayed in Grafana to help operations teams easily observe and analyze the system.

### **Làm sao phát hiện một service bị chậm trong hệ thống microservices?**

[VN] Để phát hiện một service bị chậm trong hệ thống microservices, thường cần theo dõi các metric về latency, request rate và error rate. Ví dụ nếu latency của một API tăng cao hoặc request xử lý lâu hơn bình thường thì có thể service đó đang gặp vấn đề. Các metric này thường được thu thập bởi Prometheus và hiển thị trên dashboard của Grafana. Ngoài ra có thể kiểm tra thêm CPU, memory usage, database query time hoặc message queue backlog để xác định nguyên nhân khiến service xử lý chậm.

[EN] To detect a slow service in a microservices system, it is important to monitor metrics such as latency, request rate, and error rate. For example, if the latency of an API increases or requests take longer to process than normal, the service may have a performance issue. These metrics are typically collected by Prometheus and visualized on dashboards in Grafana. In addition, engineers may check CPU usage, memory usage, database query time, or message queue backlog to identify the root cause of the slowdown.

### **Monitoring Kafka hoặc IBM MQ nên theo dõi metric nào?**

[VN] Khi monitoring hệ thống message queue như Apache Kafka hoặc IBM MQ, cần theo dõi một số metric quan trọng để đảm bảo message được xử lý ổn định. Ví dụ message rate để biết số lượng message được gửi và xử lý mỗi giây, queue size hoặc queue depth để biết có bao nhiêu message đang chờ xử lý, và consumer lag để phát hiện consumer có đang xử lý chậm hay không. Ngoài ra cũng nên theo dõi processing time, error rate và resource usage (CPU, memory) của các consumer service để nhanh chóng phát hiện vấn đề trong hệ thống.

[EN] When monitoring message queue systems such as Apache Kafka or IBM MQ, it is important to track several key metrics to ensure messages are processed reliably. For example, message rate shows how many messages are produced and consumed per second, queue size or queue depth indicates how many messages are waiting to be processed, and consumer lag helps detect if consumers are processing messages too slowly. It is also useful to monitor processing time, error rate, and resource usage (CPU and memory) of consumer services to quickly identify system issues.

### **Làm sao debug khi transaction bị chậm trong hệ thống banking?**

[VN] Khi transaction bị chậm trong hệ thống banking, trước tiên cần kiểm tra các metric về latency, request rate và error rate để xác định service nào đang xử lý chậm. Các metric này thường được theo dõi bằng Prometheus và hiển thị trên dashboard của Grafana. Sau đó có thể kiểm tra thêm database query time, message queue backlog (Kafka hoặc MQ), CPU và memory usage của service. Ngoài ra việc xem log của service cũng giúp xác định lỗi hoặc bước xử lý nào đang gây chậm trong luồng transaction.

[EN] When a transaction becomes slow in a banking system, the first step is to check metrics such as latency, request rate, and error rate to identify which service is processing slowly. These metrics are usually monitored using Prometheus and visualized on dashboards in Grafana. After that, engineers can investigate database query time, message queue backlog (Kafka or MQ), and CPU or memory usage of the service. Checking the service logs can also help identify errors or specific steps in the transaction flow that are causing the delay.

## **Pull model của Prometheus khác gì với push model?**

[VN] Trong Prometheus, pull model nghĩa là Prometheus sẽ chủ động gửi request đến các service để lấy metric từ endpoint như /metrics theo một khoảng thời gian định kỳ. Cách này giúp Prometheus dễ quản lý việc thu thập metric, kiểm tra trạng thái service và đảm bảo dữ liệu được lấy ổn định. Ngược lại, trong push model, các service sẽ chủ động gửi metric đến hệ thống monitoring. Push model thường dùng khi service không thể expose endpoint hoặc chạy trong môi trường ngắn hạn. Tuy nhiên pull model phổ biến hơn vì đơn giản, dễ kiểm soát và dễ phát hiện service nào đang không hoạt động.

[EN] In Prometheus, the pull model means that Prometheus actively sends requests to services to collect metrics from endpoints such as /metrics at regular intervals. This approach makes it easier for Prometheus to manage metric collection, check service availability, and ensure stable monitoring. In contrast, the push model means that services actively send their metrics to a monitoring system. Push model is usually used when services cannot expose endpoints or run in short-lived environments. However, the pull model is more common because it is simpler, easier to control, and helps detect when a service is not responding.

## **Khi hệ thống scale nhiều instance (Kubernetes), Prometheus xử lý metric aggregation như thế nào?**

[VN] Khi hệ thống chạy nhiều instance trong Kubernetes, Prometheus sẽ scrape metric từ từng pod hoặc service riêng biệt. Mỗi metric sẽ có các label như instance, pod, hoặc service để phân biệt nguồn dữ liệu. Khi cần xem tổng thể hệ thống, Prometheus có thể sử dụng PromQL để thực hiện các phép aggregation như sum, avg hoặc max trên các metric có cùng label. Nhờ vậy ta có thể dễ dàng xem tổng request rate, tổng error rate hoặc latency trung bình của tất cả các instance trong hệ thống.

[EN] When a system scales to multiple instances in Kubernetes, Prometheus scrapes metrics from each pod or service separately. Each metric includes labels such as instance, pod, or service to identify where the data comes from. When engineers need a system-wide view, Prometheus uses PromQL to perform aggregation operations like sum, avg, or max across metrics with similar labels. This allows teams to see metrics such as total request rate, overall error rate, or average latency across all service instances.

## **Nếu dashboard Grafana hiển thị CPU tăng cao nhưng request không tăng, bạn sẽ điều tra gì?**

[VN] Nếu dashboard của Grafana cho thấy CPU tăng cao nhưng số lượng request không tăng, trước tiên cần kiểm tra log của service và các metric hệ thống để tìm nguyên nhân. Ví

dự có thể xảy ra memory leak, goroutine leak, vòng lặp vô hạn trong code, hoặc một task background chạy quá nhiều. Ngoài ra cũng nên kiểm tra database query chậm, message queue backlog hoặc retry logic hoạt động liên tục vì những yếu tố này có thể làm CPU tăng dù request từ người dùng không thay đổi. Các metric này thường được thu thập từ Prometheus để giúp phân tích nguyên nhân.

[EN] If a dashboard in Grafana shows high CPU usage but the request rate is not increasing, the first step is to check service logs and system metrics to find the cause. Possible reasons include memory leaks, goroutine leaks, infinite loops in the code, or heavy background tasks running continuously. It is also important to investigate slow database queries, message queue backlog, or retry logic running repeatedly, since these issues can increase CPU usage even when user requests remain stable. These metrics are usually collected by Prometheus to help analyze the problem.

### **Bạn có trực tiếp config Prometheus không?**

[VN] Trong dự án của tôi, việc cấu hình Prometheus và hạ tầng monitoring chủ yếu do DevOps team thực hiện. Tuy nhiên, với vai trò backend developer, tôi thường expose metric từ service, kiểm tra endpoint /metrics, và sử dụng dashboard trên Grafana để theo dõi các metric quan trọng như latency, error rate và request rate. Khi có sự cố production, tôi dùng các metric này để phân tích nguyên nhân và debug vấn đề trong service.

[EN] In my project, the configuration of Prometheus and the monitoring infrastructure was mainly handled by the DevOps team. However, as a backend developer, I was responsible for exposing metrics from services, verifying the /metrics endpoint, and using dashboards in Grafana to monitor important metrics such as latency, error rate, and request rate. When production issues occurred, I used these metrics to analyze the root cause and debug problems in the services.

### **Logging trong hệ thống microservices thường được thiết kế như thế nào?**

[VN] Trong hệ thống microservices, logging thường được thiết kế theo mô hình centralized logging, nghĩa là tất cả service sẽ ghi log ra stdout hoặc file, sau đó một log collector sẽ thu thập và gửi log đến hệ thống lưu trữ tập trung. Trong môi trường Kubernetes, các container thường ghi log ra stdout và công cụ như Fluentd sẽ thu thập log từ các pod. Log sau đó được gửi đến Elasticsearch để lưu trữ và có thể được tìm kiếm, phân tích thông qua Kibana. Cách này giúp dễ dàng theo dõi, tìm kiếm và debug log của nhiều service trong toàn hệ thống.

[EN] In a microservices system, logging is usually designed using a centralized logging approach, where all services write logs to stdout or log files, and a log collector gathers

and sends them to a centralized storage system. In a Kubernetes environment, containers typically write logs to stdout, and tools like Fluentd collect logs from pods. The logs are then sent to Elasticsearch for storage, and engineers can search and analyze them using Kibana. This design makes it easier to monitor, search, and debug logs across multiple services in the system.

### **Vai trò của từng thành phần trong EFK stack là gì?**

[VN] Trong EFK stack, mỗi thành phần có một vai trò riêng trong hệ thống logging. Fluentd có nhiệm vụ thu thập log từ các container hoặc pod trong hệ thống và có thể parse hoặc filter log trước khi gửi đi. Elasticsearch dùng để lưu trữ và index log, giúp việc tìm kiếm và phân tích log nhanh và hiệu quả. Cuối cùng, Kibana được dùng để tìm kiếm, phân tích và hiển thị log thông qua giao diện web, giúp developer và DevOps dễ dàng debug và theo dõi hệ thống.

[EN] In the EFK stack, each component has a specific role in the logging system. Fluentd is responsible for collecting logs from containers or pods in the system and can parse or filter the logs before sending them. Elasticsearch is used to store and index logs, allowing fast searching and analysis of log data. Finally, Kibana is used to search, analyze, and visualize logs through a web interface, making it easier for developers and DevOps teams to debug and monitor the system.

### **Fluentd thu thập log từ Kubernetes pods như thế nào?**

[VN] Trong môi trường Kubernetes, các container thường ghi log ra stdout hoặc stderr, và các log này được lưu tạm dưới dạng file trên node. Fluentd thường chạy dưới dạng DaemonSet, nghĩa là mỗi node sẽ có một Fluentd pod để thu thập log. Fluentd sẽ đọc các file log của container trên node, parse dữ liệu nếu cần, sau đó gửi log đến hệ thống lưu trữ như Elasticsearch để lưu trữ và tìm kiếm.

[EN] In a Kubernetes environment, containers usually write logs to stdout or stderr, and these logs are stored temporarily as files on the node. Fluentd typically runs as a DaemonSet, meaning each node has a Fluentd pod to collect logs. Fluentd reads the container log files from the node, parses the data if needed, and then sends the logs to a storage system such as Elasticsearch for storage and searching.

### **Tại sao Fluentd thường chạy dưới dạng DaemonSet trong Kubernetes?**

[VN] Trong Kubernetes, Fluentd thường chạy dưới dạng DaemonSet vì DaemonSet đảm bảo mỗi node trong cluster có một pod Fluentd. Điều này giúp Fluentd có thể truy cập và thu thập log của tất cả container chạy trên node đó, vì log container thường được lưu trên

filesystem của node. Cách triển khai này giúp việc thu thập log ổn định, phân tán và dễ scale khi cluster có thêm node mới.

[EN] In Kubernetes, Fluentd usually runs as a DaemonSet because a DaemonSet ensures that one Fluentd pod runs on every node in the cluster. This allows Fluentd to access and collect logs from all containers running on that node, since container logs are typically stored on the node's filesystem. This deployment model makes log collection reliable, distributed, and easy to scale when new nodes are added to the cluster.

### **Log từ container đi qua pipeline như thế nào trước khi lưu vào Elasticsearch?**

[VN] Trong hệ thống logging với EFK stack, log từ container thường đi qua một pipeline gồm nhiều bước. Trước hết, container trong Kubernetes ghi log ra stdout hoặc stderr. Sau đó Fluentd sẽ đọc các file log này từ node, thực hiện các bước như parse, filter hoặc thêm metadata (ví dụ: tên pod, namespace, service). Sau khi xử lý xong, Fluentd gửi log đến Elasticsearch để lưu trữ và index, giúp log có thể được tìm kiếm và phân tích dễ dàng.

[EN] In a logging system using the EFK stack, container logs go through several steps before being stored. First, containers in Kubernetes write logs to stdout or stderr. Then Fluentd reads these log files from the node and performs processing steps such as parsing, filtering, or adding metadata (for example: pod name, namespace, or service name). After processing, Fluentd sends the logs to Elasticsearch to store and index them, so the logs can be searched and analyzed easily.

### **Structured logging là gì? Tại sao nên dùng JSON log?**

[VN] Structured logging là cách ghi log theo định dạng có cấu trúc, trong đó mỗi log được lưu dưới dạng các field key–value thay vì chỉ là một chuỗi văn bản. Một định dạng phổ biến là JSON, vì nó giúp log dễ được parse và xử lý bởi các hệ thống logging như Elasticsearch. Khi dùng JSON log, chúng ta có thể dễ dàng tìm kiếm theo các field như service, request\_id, user\_id hoặc status. Điều này giúp việc phân tích log, debug và monitoring trên các công cụ như Kibana trở nên nhanh và chính xác hơn.

[EN] Structured logging is a way of writing logs in a structured format, where each log entry contains key–value fields instead of a single plain text message. A common format is JSON, because it allows logs to be easily parsed and processed by logging systems such as Elasticsearch. With JSON logs, we can easily search by fields like service, request\_id, user\_id, or status. This makes log analysis, debugging, and monitoring much easier and more efficient in tools like Kibana.

### **Trong một hệ thống banking transaction, log nên chứa những thông tin gì?**

[VN] Trong hệ thống banking transaction, log nên chứa các thông tin quan trọng để dễ theo dõi và debug giao dịch. Ví dụ: transaction ID, request ID, timestamp, service name, user hoặc account ID, trạng thái giao dịch (success/failed) và error message nếu có lỗi. Ngoài ra, log cũng nên có latency hoặc processing time để phát hiện giao dịch chậm. Khi sử dụng hệ thống log tập trung như Elasticsearch, các thông tin này giúp dễ dàng tìm kiếm và phân tích giao dịch thông qua công cụ như Kibana.

[EN] In a banking transaction system, logs should include important information to help track and debug transactions. For example: transaction ID, request ID, timestamp, service name, user or account ID, transaction status (success/failed), and error message if any failure occurs. Logs should also include latency or processing time to detect slow transactions. When using a centralized logging system like Elasticsearch, this information helps engineers easily search and analyze transactions using tools such as Kibana.

### **Làm sao tìm log của một request cụ thể trong hệ thống microservices?**

[VN] Trong hệ thống microservices, cách phổ biến để tìm log của một request cụ thể là sử dụng request ID hoặc correlation ID. Khi một request đi qua nhiều service, hệ thống sẽ tạo hoặc truyền cùng một request ID trong header của request. Mỗi service khi ghi log sẽ ghi kèm request ID đó. Sau đó ta có thể tìm tất cả log liên quan đến request này trong hệ thống logging tập trung như Elasticsearch và xem chúng trên giao diện Kibana. Cách này giúp theo dõi toàn bộ đường đi của request qua nhiều service để debug dễ dàng hơn.

[EN] In a microservices system, a common way to find logs for a specific request is by using a request ID or correlation ID. When a request travels through multiple services, the system creates or passes the same request ID in the request header. Each service includes this request ID when writing logs. Then we can search for all related logs in a centralized logging system such as Elasticsearch and view them in Kibana. This approach helps track the full path of a request across multiple services and makes debugging much easier.

### **Khi một service bị lỗi production, bạn debug bằng log như thế nào?**

[VN] Khi một service bị lỗi trong production, tôi thường bắt đầu bằng cách tìm log theo thời gian xảy ra lỗi hoặc theo request ID / transaction ID. Sau đó tôi kiểm tra các error log, stack trace và message lỗi để hiểu nguyên nhân. Tiếp theo, tôi xem log của các service liên quan để theo dõi luồng request giữa các service. Trong hệ thống logging tập trung như Elasticsearch, tôi có thể tìm kiếm log nhanh theo các field như service name, status hoặc error type và phân tích chúng trên giao diện Kibana để xác định chính xác nơi lỗi xảy ra.

[EN] When a service fails in production, I usually start by searching logs based on the time of the error or using a request ID / transaction ID. Then I check the error logs, stack traces,

and error messages to understand the root cause. After that, I review logs from related services to follow the request flow across services. In a centralized logging system like Elasticsearch, I can quickly search logs by fields such as service name, status, or error type and analyze them through the interface of Kibana to identify exactly where the failure occurred.

### **Index trong Elasticsearch là gì? Log được lưu như thế nào?**

[VN] Trong Elasticsearch, index là nơi dùng để lưu trữ và tổ chức dữ liệu, tương tự như một database table trong hệ quản trị cơ sở dữ liệu. Mỗi log được lưu dưới dạng một document (thường là JSON) trong index. Khi log được gửi từ hệ thống logging như Fluentd, Elasticsearch sẽ index các field trong document để có thể tìm kiếm nhanh theo các thuộc tính như timestamp, service name hoặc level. Nhờ vậy chúng ta có thể dễ dàng truy vấn và phân tích log thông qua các công cụ như Kibana.

[EN] In Elasticsearch, an index is a place used to store and organize data, similar to a database table in a traditional database system. Each log entry is stored as a document (usually in JSON format) inside the index. When logs are sent from a logging system such as Fluentd, Elasticsearch indexes the fields in the document so they can be searched quickly by attributes like timestamp, service name, or log level. This allows users to easily query and analyze logs using tools such as Kibana.

### **Log rotation và retention nên được thiết kế ra sao trong production?**

[VN] Trong production, log rotation và log retention cần được thiết kế để tránh việc log chiếm quá nhiều dung lượng lưu trữ. Log rotation nghĩa là hệ thống sẽ tự động tạo file log mới theo thời gian hoặc theo dung lượng, ví dụ mỗi ngày hoặc khi file đạt một kích thước nhất định. Log retention là chính sách giữ log trong một khoảng thời gian nhất định (ví dụ 7, 30 hoặc 90 ngày) rồi tự động xóa hoặc archive log cũ. Trong hệ thống logging tập trung như Elasticsearch, việc quản lý retention thường được thực hiện bằng các policy để tự động xóa hoặc chuyển index cũ, giúp hệ thống ổn định và tối ưu chi phí lưu trữ.

[EN] In production, log rotation and log retention should be designed to prevent logs from using too much storage space. Log rotation means the system automatically creates new log files based on time or file size, for example daily or when a file reaches a certain size. Log retention is a policy that keeps logs for a specific period of time (for example 7, 30, or 90 days) and then automatically deletes or archives older logs. In centralized logging systems such as Elasticsearch, retention is usually managed through policies that automatically delete or move old indexes, helping keep the system stable and optimize storage cost.

## **Làm sao giảm kích thước log để tránh Elasticsearch quá tải?**

[VN] Để giảm kích thước log và tránh quá tải cho Elasticsearch, có thể áp dụng một số cách. Thứ nhất, chỉ ghi những log cần thiết và tránh log quá nhiều thông tin debug trong production. Thứ hai, dùng structured logging và chỉ giữ các field quan trọng như timestamp, service name, request ID và error message. Thứ ba, cấu hình log level phù hợp (ví dụ chỉ dùng INFO, WARN, ERROR trong production). Ngoài ra, có thể dùng Fluentd để filter hoặc drop những log không cần thiết trước khi gửi vào Elasticsearch, giúp giảm dung lượng lưu trữ và tăng hiệu năng hệ thống.

[EN] To reduce log size and prevent overload in Elasticsearch, several strategies can be applied. First, log only necessary information and avoid excessive debug logs in production. Second, use structured logging and keep only important fields such as timestamp, service name, request ID, and error message. Third, configure an appropriate log level (for example INFO, WARN, and ERROR in production). In addition, tools like Fluentd can filter or drop unnecessary logs before sending them to Elasticsearch, which helps reduce storage usage and improve system performance.

## **Log level (INFO, WARN, ERROR, DEBUG) nên dùng khi nào?**

[VN] Trong hệ thống logging, các log level giúp phân loại mức độ quan trọng của log. DEBUG dùng để ghi thông tin chi tiết phục vụ debug khi phát triển hoặc điều tra lỗi. INFO dùng cho các sự kiện bình thường của hệ thống như service start, request được xử lý hoặc transaction thành công. WARN dùng khi có tình huống bất thường nhưng hệ thống vẫn hoạt động, ví dụ retry hoặc response chậm. ERROR dùng khi có lỗi thực sự khiến một chức năng hoặc request thất bại. Khi sử dụng hệ thống logging tập trung như Elasticsearch và xem log qua Kibana, việc phân chia log level giúp dễ dàng lọc và tìm các lỗi quan trọng trong production.

[EN] In a logging system, log levels help classify the importance of log messages. DEBUG is used for detailed information mainly for debugging during development or troubleshooting. INFO is used for normal system events such as service start, request processing, or successful transactions. WARN is used when something unusual happens but the system can still continue running, for example retries or slow responses. ERROR is used when a real failure occurs and a function or request cannot be completed. When using centralized logging systems like Elasticsearch and viewing logs in Kibana, log levels help engineers quickly filter and find important issues in production.

## **Nếu Elasticsearch bị chậm hoặc disk gần đầy thì ảnh hưởng gì đến hệ thống logging?**

[VN] Nếu Elasticsearch bị chậm hoặc disk gần đầy, hệ thống logging có thể bị ảnh hưởng đáng kể. Log từ các service được gửi qua collector như Fluentd có thể bị chậm ghi hoặc bị

queue lại, và trong một số trường hợp log mới có thể bị drop nếu buffer đầy. Điều này làm cho việc theo dõi hệ thống và debug production trở nên khó khăn vì một số log quan trọng có thể không được lưu lại. Ngoài ra, việc tìm kiếm log trên các công cụ như Kibana cũng sẽ chậm hơn do Elasticsearch xử lý truy vấn kém hiệu quả.

[EN] If Elasticsearch becomes slow or the disk is almost full, the logging system can be significantly affected. Logs sent from services through collectors such as Fluentd may be written slowly or queued, and in some cases new logs may be dropped if the buffer becomes full. This makes system monitoring and production debugging more difficult because some important logs may not be stored. In addition, searching logs through tools like Kibana will also become slower due to reduced Elasticsearch query performance.

### **Làm sao liên kết log với monitoring metrics để debug hệ thống?**

[VN] Để liên kết log với monitoring metrics khi debug hệ thống, chúng ta thường sử dụng các thông tin chung như timestamp, service name, request ID hoặc transaction ID. Ví dụ, khi thấy một metric như latency hoặc error rate tăng trên Prometheus và dashboard Grafana, chúng ta có thể kiểm tra log của service trong cùng khoảng thời gian đó. Sau đó tìm theo request ID hoặc transaction ID trong hệ thống log tập trung như Elasticsearch để xác định request nào gây lỗi hoặc chậm. Cách này giúp kết nối thông tin từ metric và log để tìm nguyên nhân gốc của vấn đề.

[EN] To link logs with monitoring metrics when debugging a system, we usually use common information such as timestamp, service name, request ID, or transaction ID. For example, if we see a metric like latency or error rate increasing in Prometheus and dashboards in Grafana, we can check logs from the same service during the same time period. Then we search by request ID or transaction ID in a centralized logging system such as Elasticsearch to identify which request caused the failure or slowdown. This approach connects metrics and logs to help find the root cause of system issues.

### **[VN] CloudWatch dùng để làm gì?**

#### **[EN] What is CloudWatch used for?**

[VN] CloudWatch là dịch vụ giám sát của Amazon Web Services dùng để theo dõi metric, log và sự kiện của hệ thống trên cloud. Nó giúp monitor CPU, memory, network, trạng thái EC2, Lambda, RDS và nhiều dịch vụ khác. CloudWatch cho phép tạo alarm khi vượt ngưỡng như CPU cao hoặc lỗi nhiều, từ đó gửi thông báo hoặc tự động scale. Nhờ đó, hệ thống có thể được theo dõi liên tục, phát hiện sự cố sớm và phản ứng kịp thời để đảm bảo ổn định.

[EN] CloudWatch is a monitoring service of Amazon Web Services used to track metrics, logs, and events in cloud systems. It helps monitor CPU, memory, network, and the status of services like EC2, Lambda, and RDS. CloudWatch allows you to create alarms when thresholds are exceeded, such as high CPU or many errors, and can send notifications or trigger auto scaling. This enables continuous monitoring, early issue detection, and quick response to keep the system stable.

**[VN] Thiết kế hệ thống logging tập trung cho microservices.**

**[EN] Design a centralized logging system for microservices.**

[VN] Để thiết kế hệ thống logging tập trung cho microservices, tôi đảm bảo mỗi service ghi log theo format thống nhất (ví dụ JSON) và không lưu log cục bộ lâu dài trong container. Log sẽ được ghi ra stdout và được thu thập bởi agent như Fluent Bit hoặc Logstash, sau đó gửi về hệ thống tập trung như Elasticsearch hoặc Cloud logging. Tôi phân loại log theo level như INFO, WARN, ERROR và thêm correlation ID hoặc trace ID vào mỗi request để có thể theo dõi xuyên suốt nhiều service. Hệ thống logging cần có khả năng search, filter và tạo dashboard để hỗ trợ debug và monitoring. Cách này giúp dễ phân tích sự cố, theo dõi hiệu năng và kiểm tra hành vi hệ thống trong môi trường production.

[EN] To design centralized logging for microservices, I ensure each service writes logs in a consistent format (for example JSON) and does not store logs locally in containers for a long time. Logs are written to stdout and collected by an agent like Fluent Bit or Logstash, then sent to a centralized system such as Elasticsearch or cloud logging. I classify logs by level such as INFO, WARN, and ERROR, and include correlation ID or trace ID in each request to track it across multiple services. The logging system must support search, filtering, and dashboards to help with debugging and monitoring. This approach makes it easier to analyze incidents, monitor performance, and understand system behavior in production.

**[VN] Cách logging chuẩn trong production?**

**[EN] How to implement proper logging in production?**

[VN] Logging chuẩn trong production nên dùng module logging thay vì print, cấu hình level rõ ràng như DEBUG, INFO, WARNING, ERROR và chỉ bật DEBUG khi cần. Nên dùng structured logging (ví dụ JSON) để dễ tìm kiếm và phân tích log trên hệ thống như ELK hoặc Cloud logging. Log cần có thông tin quan trọng như timestamp, request id, user id (nếu phù hợp) để trace lỗi. Không log thông tin nhạy cảm như mật khẩu hoặc token. Ngoài ra, nên cấu hình log rotation và gửi log tập trung để dễ monitor và alert khi có lỗi nghiêm trọng.

[EN] Proper logging in production should use the logging module instead of print, with clear log levels such as DEBUG, INFO, WARNING, and ERROR, and only enable DEBUG when necessary. It is recommended to use structured logging (for example JSON format) to make searching and analyzing logs easier in systems like ELK or cloud logging platforms. Logs should include important information such as timestamp, request ID, and user ID (if appropriate) to help trace issues. Sensitive data like passwords or tokens should never be logged. You should also configure log rotation and centralize logs for monitoring and alerting on critical errors.

**[VN] Structured logging là gì?**

**[EN] What is structured logging?**

[VN] Structured logging là cách ghi log theo định dạng có cấu trúc như JSON thay vì chỉ là chuỗi text tự do. Mỗi log sẽ gồm các field rõ ràng như timestamp, level, message, request\_id, user\_id... giúp hệ thống log có thể tìm kiếm, lọc và phân tích dễ dàng bằng công cụ như ELK hoặc cloud logging. Cách này giúp việc debug, monitor và trace lỗi trong production nhanh và chính xác hơn, đặc biệt khi hệ thống có nhiều service.

[EN] Structured logging is a way of writing logs in a structured format such as JSON instead of plain free-text strings. Each log entry contains clear fields like timestamp, level, message, request\_id, user\_id, and more, making it easier for logging systems to search, filter, and analyze using tools like ELK or cloud logging platforms. This approach improves debugging, monitoring, and tracing in production, especially in systems with multiple services.

## **# IX. Observability & Monitoring**

**Prometheus thu thập metric từ microservices như thế nào?**

**Trong Kubernetes, Prometheus discover service để scrape metric bằng cách nào?**

**Grafana lấy dữ liệu từ Prometheus như thế nào để hiển thị dashboard?**

**Metric, log và trace khác nhau như thế nào trong monitoring system?**

**Trong một hệ thống banking transaction, bạn sẽ monitor những metric nào quan trọng?**

**Làm sao expose metric từ Golang service cho Prometheus?**

**Histogram và Counter trong Prometheus dùng khi nào?**

**Alerting trong Prometheus hoạt động như thế nào?**

**Grafana dashboard thường dùng để visualize những metric gì trong production?**

**Làm sao phát hiện một service bị chậm trong hệ thống microservices?**

**Monitoring Kafka hoặc IBM MQ nên theo dõi metric nào?**

**Làm sao debug khi transaction bị chậm trong hệ thống banking?**

**Pull model của Prometheus khác gì với push model?**

**Khi hệ thống scale nhiều instance (Kubernetes), Prometheus xử lý metric aggregation như thế nào?**

**Nếu dashboard Grafana hiển thị CPU tăng cao nhưng request không tăng, bạn sẽ điều tra gì?**

**Bạn có trực tiếp config Prometheus không?**

**Logging trong hệ thống microservices thường được thiết kế như thế nào?**

**Vai trò của từng thành phần trong EFK stack là gì?**

**Fluentd thu thập log từ Kubernetes pods như thế nào?**

**Tại sao Fluentd thường chạy dưới dạng DaemonSet trong Kubernetes?**

**Log từ container đi qua pipeline như thế nào trước khi lưu vào Elasticsearch?**

**Structured logging là gì? Tại sao nên dùng JSON log?**

**Trong một hệ thống banking transaction, log nên chứa những thông tin gì?**

**Làm sao tìm log của một request cụ thể trong hệ thống microservices?**

**Khi một service bị lỗi production, bạn debug bằng log như thế nào?**

**Index trong Elasticsearch là gì? Log được lưu như thế nào?**

**Log rotation và retention nên được thiết kế ra sao trong production?**

**Làm sao giảm kích thước log để tránh Elasticsearch quá tải?**

**Log level (INFO, WARN, ERROR, DEBUG) nên dùng khi nào?**

**Nếu Elasticsearch bị chậm hoặc disk gần đầy thì ảnh hưởng gì đến hệ thống logging?**

**Làm sao liên kết log với monitoring metrics để debug hệ thống?**

**CloudWatch dùng để làm gì?**

**Thiết kế hệ thống logging tập trung cho microservices.**

**Cách logging chuẩn trong production?**

**Structured logging là gì?**